FIAP · Faculdade de Informática e Administração Paulista
Análise e Desenvolvimento de Sistemas · 1º ano

MOBILIDADE VERDE

API Restful de resgate de Pontos ECOA
em cupons de transporte público

EQUIPE FOG

Integrante	RM	Função
Artur Bianchini Esteves	RM569450	**Representante**
Diego Barbosa Rodrigues	RM568829	—
João Vitor Oliveira Fontenele Martins	RM570783	—
Vinicius Pacheco Ruiz	RM571109	—
Yan de Almeida Cardoso	RM568814	—

Turma **1TDSPX-2026**
Disciplina: **Domain Driven Design using Java**
Entrega da **Sprint 4** — Challenge 2026 · SoulUp / Prospera

São Paulo · 2026

Sumário

1. Objetivo e escopo do projeto 4

1.1 Propósito do sistema 4

1.2 O que está dentro do escopo 4

1.3 O que está fora do escopo 4

1.4 Regras de negócio implementadas 5

1.5 Parâmetros oficiais do projeto 5

2. Principais funcionalidades da solução 6

2.1 Funcionalidades de domínio 6

2.2 Funcionalidades de CRUD 6

2.3 Segurança e integridade aplicadas 7

3. Tabela de endpoints da API Restful 8

3.1 Operadoras de transporte 8

3.2 Faixas de cupom 8

3.3 Usuários 8

3.4 Vouchers (cupons) 9

3.5 Status da aplicação 9

3.6 Códigos de status de resposta 9

3.7 Exemplos de requisição e resposta 9

4. Protótipo — prints das telas implementadas 11

5. Modelo de Entidade-Relacionamento (MER) 18

5.1 As oito tabelas 19

5.2 Decisões de modelagem que sustentam a API 20

6. Diagrama de classes atualizado 21

6.1 Visão geral da arquitetura 21

6.2 Camada Model — as classes de domínio 23

6.3 Camada DAO e a Connection Factory 25

6.4 Camada BO — as regras de negócio 27

6.5 Camada Resource, CORS e tratamento de erros 28

6.6 Organização dos pacotes 29

7. Procedimentos para rodar a aplicação 30

7.1 Ferramentas e versões 30

7.2 Pré-requisitos no banco 30

7.3 Execução pelo terminal 30

7.4 Importação no IntelliJ IDEA 30

7.5 Credenciais de acesso ao banco 31

8. Deploy da aplicação 32

8.1 Links da entrega 32

8.2 Como o serviço foi configurado 32

8.3 Como validar que o deploy está funcionando 32

8.4 Estrutura do projeto entregue 33

1. Objetivo e escopo do projeto

1.1 Propósito do sistema

O **Mobilidade Verde** é a funcionalidade da plataforma SoulUp que transforma **Pontos ECOA** — a moeda verde que o usuário já acumula assistindo anúncios, participando de campanhas e comprando em parceiros — em **cupons de transporte público** aceitos por operadoras parceiras. O usuário escolhe a operadora e a faixa de valor, o sistema debita os pontos da carteira e devolve um código de cupom com validade de 24 horas. Quando o cupom é usado no embarque, o sistema registra a viagem e calcula o CO_2 evitado por ter trocado o carro pelo transporte coletivo.

Nesta Sprint 4 o sistema deixa de ser uma aplicação de console e passa a ser uma **API Restful em Spring Boot**, publicada na nuvem e consumida pelo front-end da equipe. Todo o domínio construído nas sprints anteriores — carteira, catálogo de operadoras, faixas de cupom, vouchers, viagens, extrato de pontos e favoritos — passa a ser exposto por endpoints HTTP com verbos, status codes e corpo JSON.

1.2 O que está dentro do escopo

- **CRUD completo de usuários**, com criação automática da carteira de pontos e remoção em cascata controlada pela aplicação.
- **CRUD completo de operadoras de transporte** (catálogo de parceiras).
- **Consulta ao catálogo de faixas de cupom** e às cotações históricas.
- **Resgate de pontos em cupom**, com débito transacional da carteira, geração de código, hash de validação e registro no extrato.
- **Utilização e estorno de cupom**, com registro de viagem e devolução de pontos.
- **Rotina de expiração em lote** dos cupons vencidos.
- **Relatórios**: impacto ambiental por usuário, ranking de CO_2 evitado e desempenho de aproveitamento por operadora.
- **Favoritar operadoras** — o relacionamento N:N do modelo de dados.
- **Endpoint de diagnóstico** que informa se a aplicação subiu e se o banco Oracle está alcançável.

1.3 O que está fora do escopo

- **Autenticação e autorização**. A API é aberta nesta entrega; o usuário é identificado pelo `id` enviado na requisição. O campo de senha já é armazenado como hash e o CPF nunca é gravado em claro, preparando o terreno para o login nas próximas etapas.
- **Integração real com a bilhetagem** das operadoras. O cupom é gerado e validado pelo próprio sistema; a troca com os sistemas de bilhetagem é uma evolução prevista, não implementada.
- **Crédito de Pontos ECOA**. A entrada de pontos acontece na plataforma SoulUp; aqui o saldo é consumido e, no estorno, devolvido.
- **Pagamento e emissão fiscal**, que pertencem ao ecossistema SoulUp.

1.4 Regras de negócio implementadas

As regras abaixo estão garantidas em dois níveis: por restrição no banco de dados (o que impede qualquer caminho de gravar dado inconsistente) e por validação na camada BO (o que devolve ao front uma mensagem explicativa em vez de um erro de banco).

RN	Regra	Onde é garantida
RN01	Todo usuário tem exatamente uma carteira, criada junto com o cadastro	UK_CARTEIRA_USUARIO + UsuarioBO.cadastrar
RN02	O saldo de pontos nunca fica negativo	CK_CARTEIRA_SALDO + Carteira.debitar()
RN03	O cupom só pode ser gerado em faixa de valor fixa e ativa	FK_VOUCHER_FAIXA + ConversorBO
RN04	Cada cupom é de uso único e expira em 24 horas	CK_VOUCHER_STATUS + CupomBO
RN05	Toda movimentação de pontos é registrada e o registro nunca é alterado	TransacaoDAO sem update de negócio
RN06	O cupom só pode ser gerado para operadora ativa	CK_OPERADORA_ATIVO + ConversorBO
RN07	Cada usuário mantém no máximo 5 cupons simultâneos no status GERADO	AntiFraudeBO + IDX_VOUCHER_USR_STAT

1.5 Parâmetros oficiais do projeto

Parâmetro	Valor adotado	Origem
Cotação dos Pontos ECOA	100 pontos = R$ 1,00	Definição do produto
Faixas de cupom ativas	R$ 2 · R$ 5 · R$ 10 · R$ 20	Catálogo T_FAIXA_CUPOM
Validade do cupom	24 horas a partir da geração	RN04
Emissão de CO_2 — carro	0,180 kg por km	Ministério do Meio Ambiente
Emissão de CO_2 — ônibus	0,082 kg por km por passageiro	ANTP
CO_2 evitado por km	0,098 kg	Diferença entre os dois acima
Km médio por passagem	7 km (São Paulo)	Pesquisa OD / Metrô-SP
Absorção de uma árvore	21,77 kg de CO_2 por ano	Literatura de reflorestamento

2. Principais funcionalidades da solução

As funcionalidades abaixo estão todas implementadas e expostas por endpoints. A coluna da direita indica a classe que concentra a regra.

2.1 Funcionalidades de domínio

Funcionalidade	O que faz	Classe responsável
Resgatar pontos em cupom	Valida saldo, faixa ativa, operadora ativa e limite antifraude; debita a carteira, gera o código e o hash de validação, grava o voucher e registra o débito no extrato. Tudo em uma única transação JDBC: se qualquer passo falhar, nada é gravado.	ConversorBO
Utilizar cupom no embarque	Confere se o cupom existe, se pertence à operadora informada, se ainda está no status GERADO e se não expirou. Marca como UTILIZADO e registra a viagem com o CO_2 evitado calculado.	CupomBO
Estornar cupom	Cancela um cupom ainda não utilizado e devolve os pontos à carteira, registrando um crédito de origem ESTORNO no extrato.	ConversorBO
Expirar cupons vencidos	Rotina em lote que varre os cupons GERADO com data de expiração no passado e os move para EXPIRADO.	CupomBO
Controle antifraude	Limita a 5 os cupons simultâneos no status GERADO por usuário e informa quantas vagas restam.	AntiFraudeBO
Impacto ambiental acumulado	Soma as viagens do usuário e devolve km percorridos, CO_2 evitado em kg e o equivalente em árvores-ano e árvores-dia.	ImpactoBO
Ranking de impacto	Classifica os usuários pelo CO_2 evitado acumulado, somando viagens por usuário com três junções no banco.	ViagemDAO.rankingImpacto()
Desempenho por operadora	Mostra, por parceira, quantos cupons foram gerados e qual o percentual efetivamente utilizado.	OperadoraDAO.desempenho()
Operadoras favoritas	Permite favoritar e desfavoritar operadoras, com ordem de preferência — é o relacionamento N:N do modelo.	OperadoraBO

2.2 Funcionalidades de CRUD

O CRUD completo (Create, Read, Update e Delete) está implementado para as duas entidades que o front precisa manter: **usuário** e **operadora de transporte**. As demais entidades são lidas e manipuladas pelos casos de uso de domínio — um voucher, por exemplo, não é editado diretamente: ele nasce de um resgate e muda de status por utilização, estorno ou expiração, que é o comportamento correto para um documento de valor.

Operação	Usuário	Operadora
Create	Cadastra o usuário, gera o hash da senha e do CPF e cria a carteira com o saldo inicial, tudo na mesma transação	Cadastra a operadora parceira com modal, valor da passagem e fator de CO_2
Read	Lista todos, busca por id (já com o saldo da carteira) e expõe cupons, extrato, impacto e favoritas	Lista as ativas, lista todas (inclusive desativadas), busca por id e relatório de desempenho
Update	Atualiza nome, e-mail e cidade; a senha é regravada apenas quando enviada	Atualiza todos os campos do catálogo, inclusive o indicador de ativo

Operação	Usuário	Operadora
Delete	Remove o usuário e todas as suas dependências na ordem correta, em transação única	Remove do catálogo; se houver cupom vinculado, a chave estrangeira impede e a API responde 500 com a explicação

2.3 Segurança e integridade aplicadas

- **Senhas e CPF nunca trafegam nem ficam em claro.** A senha é gravada como hash SHA-256 e o CPF como hash; o DTO de resposta (`UsuarioResponse`) existe justamente para que esses dois campos nunca saiam da API.
- **Consultas sempre parametrizadas.** Todo acesso ao banco usa `PreparedStatement`, o que elimina a superfície de SQL Injection.
- **Transações explícitas.** Operações que tocam mais de uma tabela abrem a conexão com `setAutoCommit(false)` e fazem `commit` ou `rollback`.
- **Cupom assinado.** Cada voucher carrega um hash HMAC-SHA256 que permite conferir, na validação, que o código não foi forjado.
- **Conexão configurável por ambiente.** URL, usuário e senha do Oracle podem ser definidos pelas variáveis `ORACLE_URL`, `ORACLE_USER` e `ORACLE_PASSWORD`, o que permite apontar a aplicação para outro schema sem recompilar — é assim que o serviço na nuvem é configurado. Os valores padrão gravados na classe de conexão são os do schema RM569450, mantidos de propósito para que o avaliador consiga clonar o repositório e executar o projeto sem nenhuma configuração prévia, como a Sprint 3 exigia.
- **Rastreabilidade.** O extrato de pontos é append-only e guarda a origem e o IP de cada movimentação.

3. Tabela de endpoints da API Restful

Todos os endpoints estão sob o prefixo `/api` e trocam JSON. A API segue os princípios REST: o recurso está na URI, a ação está no verbo HTTP e o resultado está no status code.

3.1 Operadoras de transporte

Verbo	URI	Descrição	Status
GET	`/api/operadoras`	Lista as operadoras ativas	200
GET	`/api/operadoras/todas`	Lista todas, inclusive as desativadas	200
GET	`/api/operadoras/desempenho`	Relatório de cupons gerados e aproveitamento por parceira	200
GET	`/api/operadoras/{id}`	Detalha uma operadora	200 · 404
POST	`/api/operadoras`	Cadastra uma parceira	201 · 400
PUT	`/api/operadoras/{id}`	Atualiza os dados da parceira	200 · 400 · 404
DELETE	`/api/operadoras/{id}`	Remove a parceira do catálogo	204 · 404 · 500

3.2 Faixas de cupom

Verbo	URI	Descrição	Status
GET	`/api/faixas`	Lista as faixas de valor ativas	200
GET	`/api/faixas/todas`	Lista todas, inclusive as cotações históricas	200
GET	`/api/faixas/{id}`	Detalha uma faixa	200 · 404

3.3 Usuários

Verbo	URI	Descrição	Status
GET	`/api/usuarios`	Lista os usuários sem expor dados sensíveis	200
GET	`/api/usuarios/ranking-impacto`	Ranking de usuários por CO_2 evitado	200
GET	`/api/usuarios/{id}`	Detalha o usuário já com o saldo da carteira	200 · 404
POST	`/api/usuarios`	Cadastra o usuário e cria a carteira	201 · 400
PUT	`/api/usuarios/{id}`	Atualiza os dados cadastrais	200 · 400 · 404
DELETE	`/api/usuarios/{id}`	Remove o usuário e suas dependências	204 · 404
GET	`/api/usuarios/{id}/vouchers`	Lista os cupons do usuário	200 · 404
GET	`/api/usuarios/{id}/transacoes`	Extrato de movimentação de pontos	200 · 404
GET	`/api/usuarios/{id}/impacto`	Impacto ambiental acumulado	200 · 404
GET	`/api/usuarios/{id}/favoritas`	Lista as operadoras favoritas (N:N)	200 · 404
POST	`/api/usuarios/{id}/favoritas`	Favorita uma operadora	201 · 404 · 500
DELETE	`/api/usuarios/{id}/favoritas/{idFavorita}`	Remove a operadora dos favoritos	204 · 404

3.4 Vouchers (cupons)

Verbo	URI	Descrição	Status
POST	`/api/vouchers`	Resgata pontos e gera o cupom	201 · 400 · 404 · 409
GET	`/api/vouchers/{codigo}`	Consulta um cupom pelo código	200 · 404
POST	`/api/vouchers/{codigo}/utilizar`	Valida o cupom no embarque e registra a viagem	200 · 404 · 409
POST	`/api/vouchers/{codigo}/estornar`	Cancela o cupom e devolve os pontos	200 · 404 · 409
POST	`/api/vouchers/expirar-vencidos`	Rotina em lote de expiração	200

3.5 Status da aplicação

Verbo	URI	Descrição	Status
GET	`/`	Identificação da API e lista de recursos	200
GET	`/api/status`	Informa se a aplicação subiu e se o Oracle está alcançável	200 · 503

3.6 Códigos de status de resposta

O tratamento é centralizado na classe `TratadorDeErros`, anotada com `@RestControllerAdvice`. Ela traduz as exceções do domínio em status HTTP, de modo que o front consiga distinguir "não encontrado" de "saldo insuficiente" sem precisar ler texto.

Código	Significado	Quando acontece nesta API
200	OK	Consulta ou atualização concluída
201	Created	Usuário, operadora, cupom ou favorita criados
204	No Content	Remoção concluída, sem corpo de resposta
400	Bad Request	Dado inválido enviado pelo cliente — campo obrigatório ausente, e-mail mal formado, valor negativo
404	Not Found	O recurso pedido não existe: id ou código de cupom inexistente
409	Conflict	Uma regra de negócio impede a operação: saldo insuficiente, limite antifraude atingido, cupom já utilizado ou expirado, operadora inativa
500	Internal Server Error	Falha de banco, incluindo violação de chave estrangeira ao tentar remover um registro ainda referenciado
503	Service Unavailable	A aplicação está no ar, mas não consegue alcançar o Oracle

3.7 Exemplos de requisição e resposta

Resgatar pontos em cupom

POST /api/vouchers

```
{
  "idUsuario": 1,
  "idOperadora": 1,
  "idFaixa": 2
}
```

201 Created

```
{
  "codigo": "TOP-A1B2-C3D4",
  "nomeOperadora": "TOP",
  "valorCentavos": 500,
  "pontosDebitados": 500,
  "expiraEm": "2026-05-25T14:30:00"
}
```


Tentativa com saldo insuficiente

409 Conflict


```
{
  "status": 409,
  "erro": "Operacao nao permitida",
  "mensagem": "Saldo insuficiente: a carteira tem 120 pontos e a faixa exige 500.",
  "momento": "2026-05-24T09:12:44"
}
```


Consultar o impacto ambiental de um usuário

GET /api/usuarios/1/impacto → **200 OK**


```
{
  "qtdViagens": 12,
  "kmEstimado": 98.0,
  "co2EvitadoKg": 9.604,
  "arvoresAno": 0.441,
  "arvoresDia": 161.0,
  "kmCarroEvitado": 98.0
}
```

4. Protótipo — prints das telas implementadas

As telas a seguir são capturas reais do front-end da equipe, construído em React + Vite + TypeScript na disciplina de Front-End Design Engineering. Cada tela corresponde a um conjunto de endpoints desta API.

Observação honesta sobre estas capturas: elas foram feitas na versão do front entregue na Sprint 3, que ainda lê dados locais. A integração com os endpoints documentados no capítulo 3 é exatamente o objeto da Sprint 4 do front-end, que consome a URL publicada na nuvem. O layout, os fluxos e os dados exibidos são os mesmos que a API devolve.

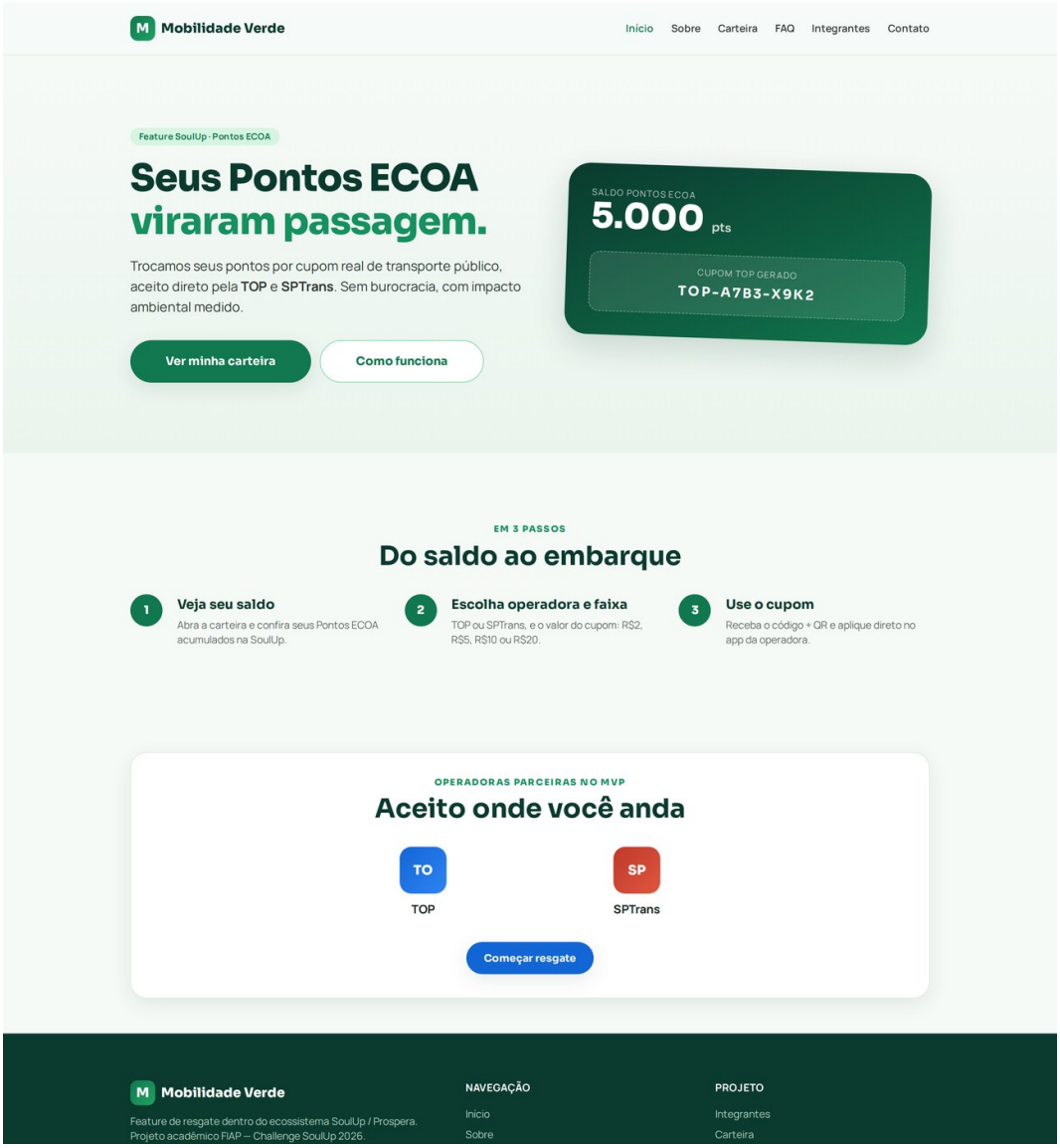


Figura 1 — Página inicial. Apresenta a proposta, mostra o saldo de Pontos ECOA e as operadoras parceiras. O card de saldo é alimentado por GET /api/usuarios/{id} e a lista de parceiras por GET /api/operadoras.

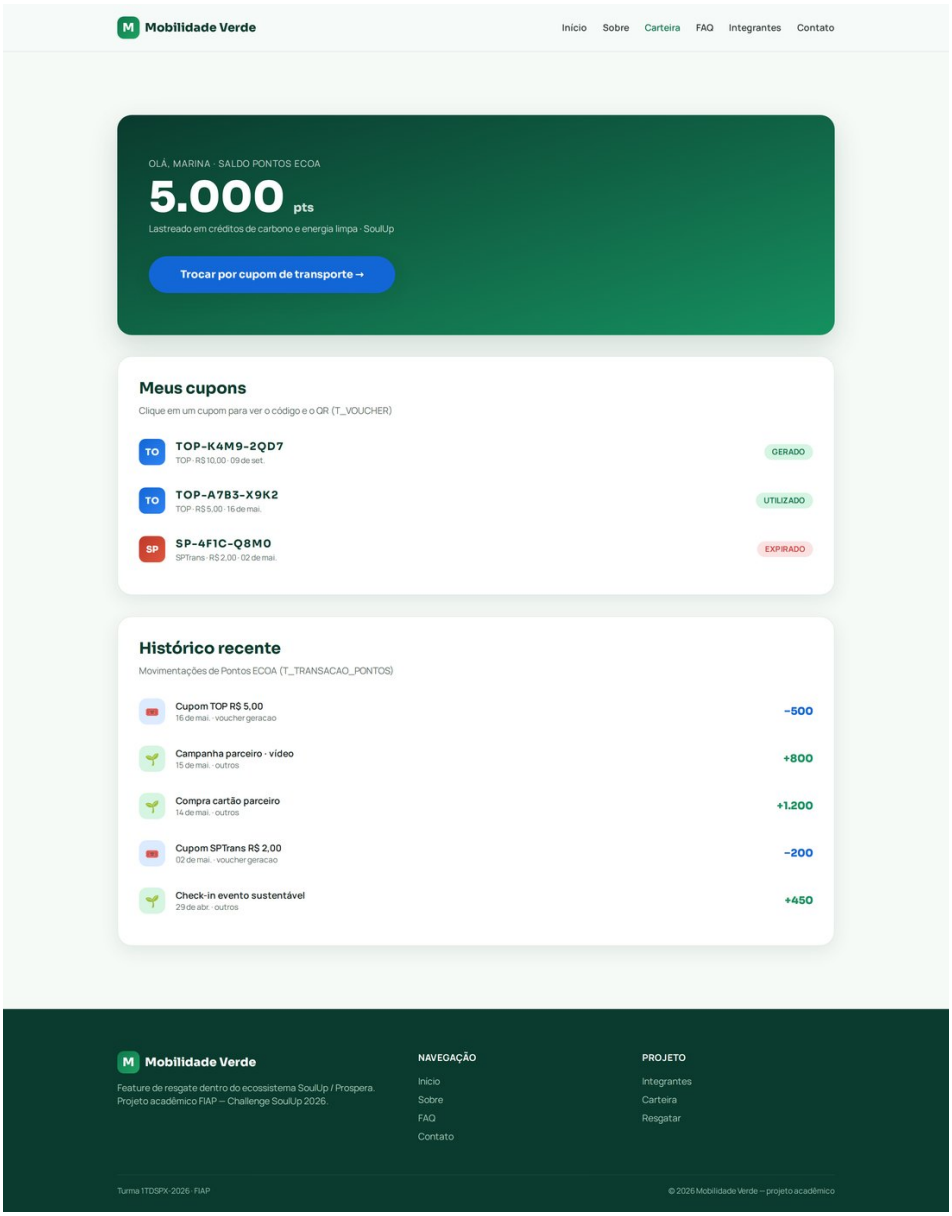


Figura 2 — Carteira. Reúne o saldo, a lista de cupons com o status de cada um (GERADO, UTILIZADO, EXPIRADO) e o histórico de movimentação de pontos. Consome GET /api/usuarios/{id}/vouchers e GET /api/usuarios/{id}/transacoes.

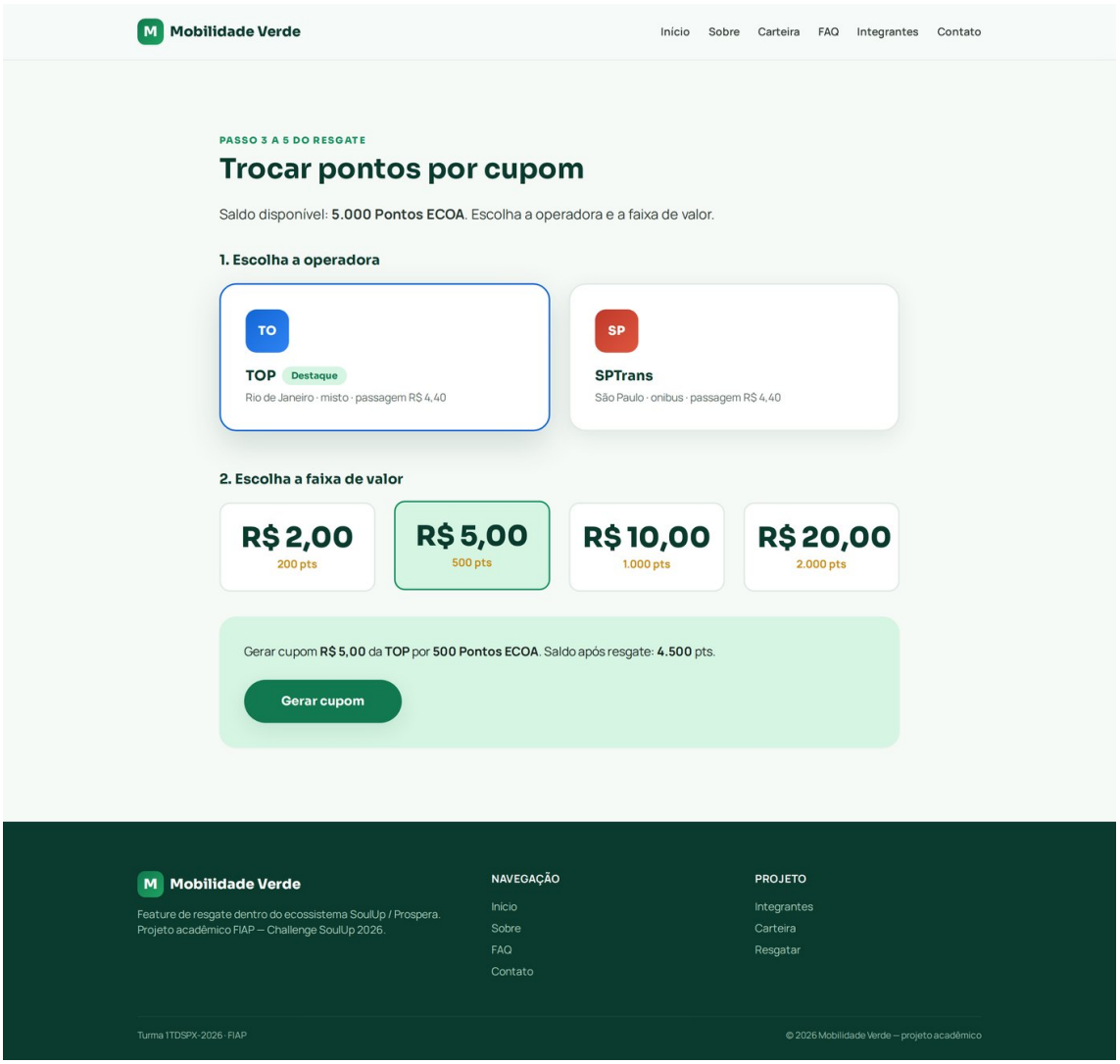


Figura 3 — Resgate. O usuário escolhe a operadora e a faixa de valor, e a tela já informa quanto sobrará de saldo. O botão Gerar cupom dispara POST /api/vouchers; a lista de faixas vem de GET /api/faixas.

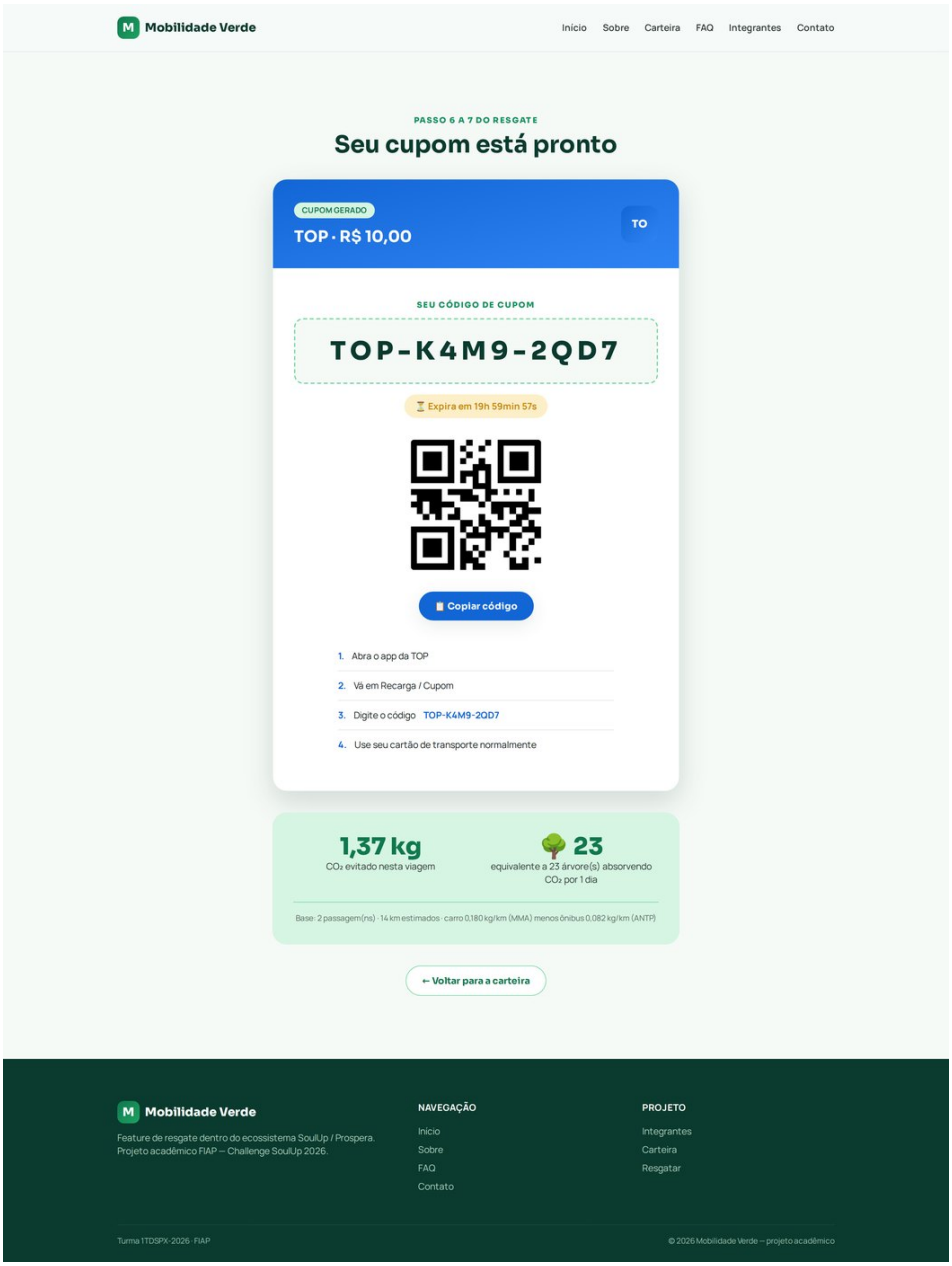


Figura 4 — Cupom gerado. Mostra o código, o QR code, o tempo restante até a expiração e o impacto ambiental daquela viagem em kg de CO_2 e em árvores. É a resposta 201 de POST /api/vouchers somada a GET /api/usuarios/{id}/impacto.

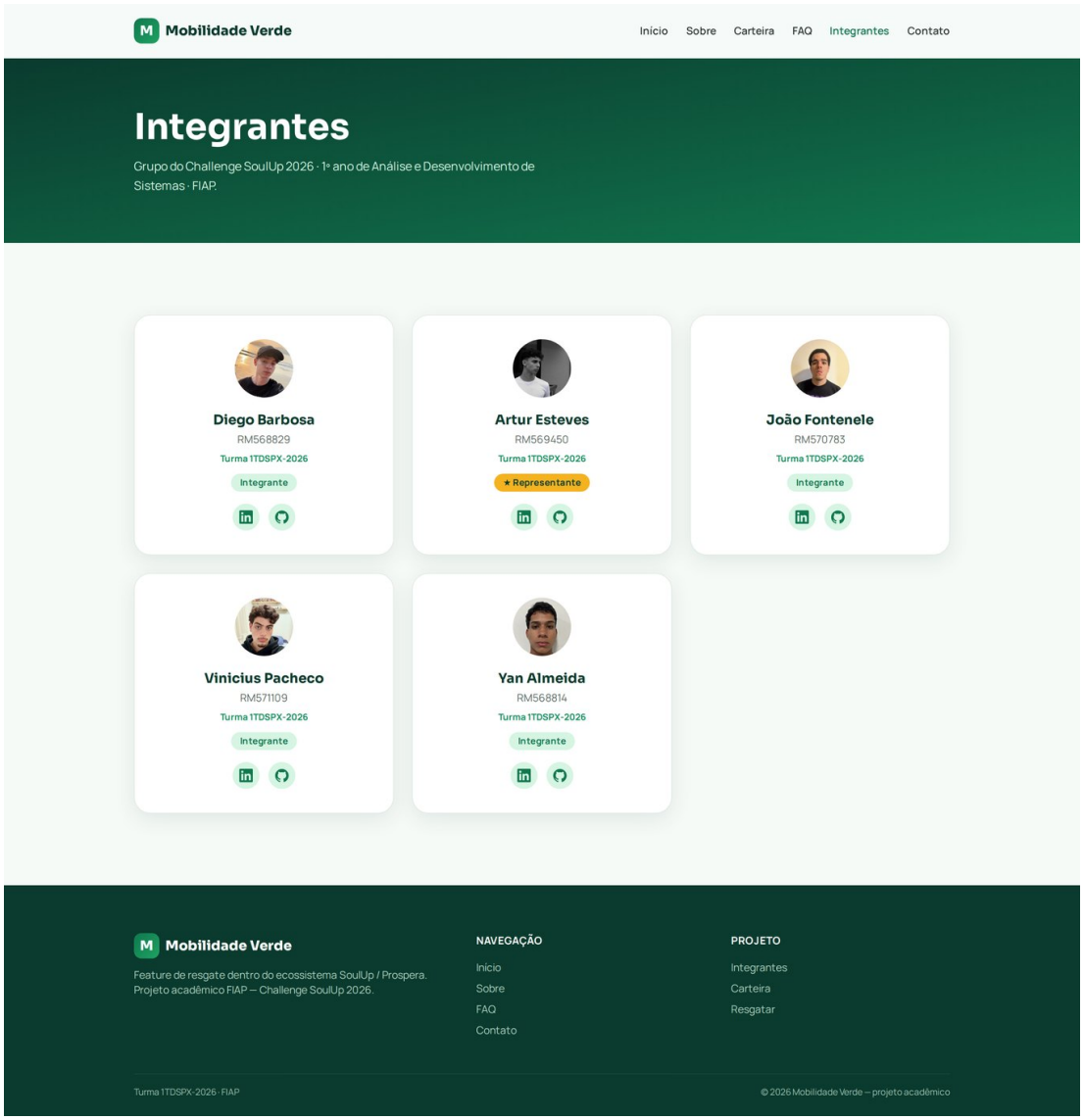


Figura 5 — Página de integrantes, com os cinco membros da Equipe FOG e seus RMs.



Figura 6 — Página Sobre. Explica o problema, a solução e o posicionamento dentro do ecossistema SoulUp / Prospera.



Figura 7 — Versão mobile da página inicial. O layout é responsivo: o menu vira hambúrguer e os blocos se empilham em coluna única.

5. Modelo de Entidade-Relacionamento (MER)

O modelo foi construído no Oracle SQL Developer Data Modeler na disciplina de Building Relational Database e é a mesma base que a API acessa. São **8 tabelas**, **8 sequences**, **5 índices** de performance e **40 restrições nomeadas** — 8 chaves primárias, 9 chaves estrangeiras, 7 restrições de unicidade e 16 verificações de domínio.

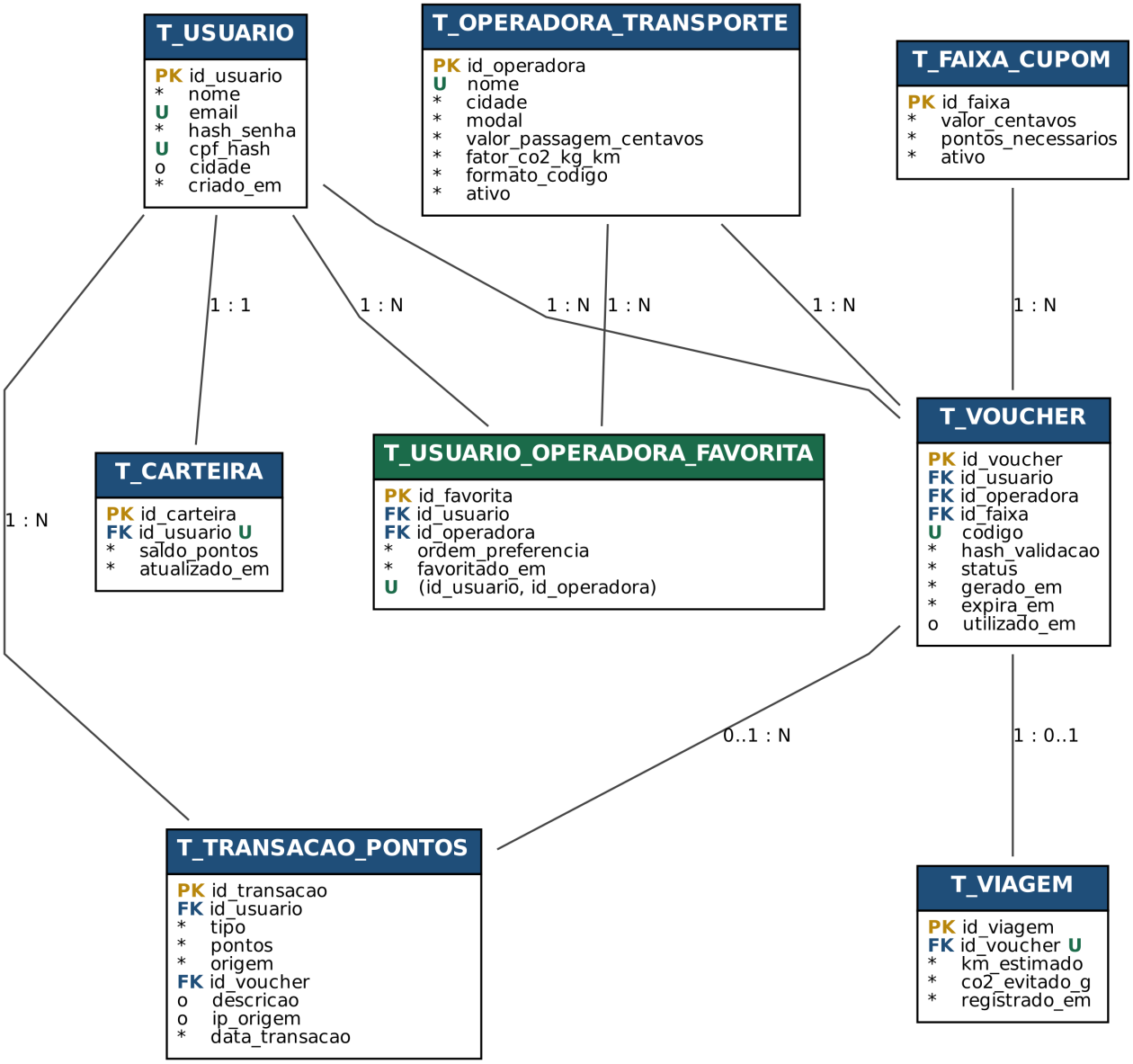


*Figura 8 — Modelo lógico. As oito entidades com seus atributos e cardinalidades. PK marca a chave primária, FK a chave estrangeira, U a restrição de unicidade, * o campo obrigatório e o o campo opcional. A entidade em verde é a associativa que resolve o N:N entre usuário e operadora.*

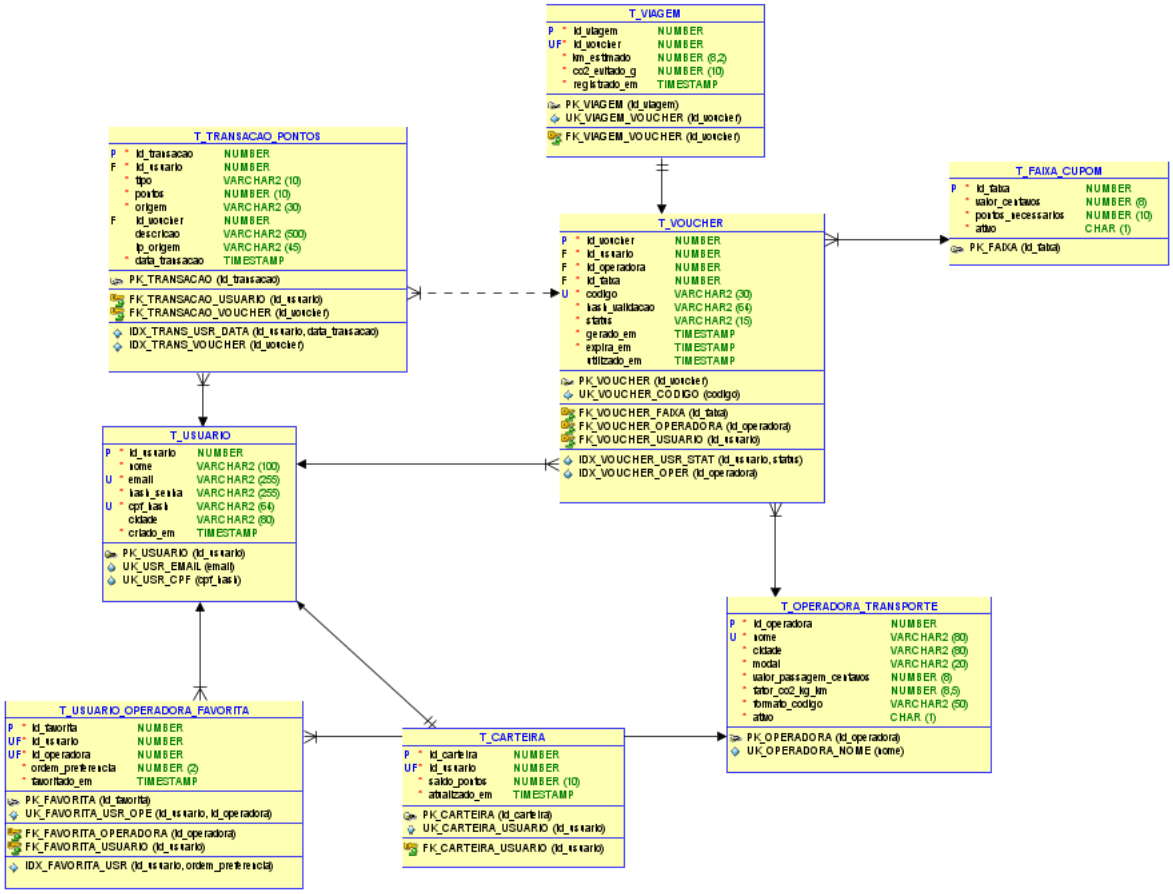


Figura 9 — Modelo físico gerado no Oracle SQL Developer Data Modeler, com os tipos de dado, as restrições e os índices efetivamente criados no banco.

5.1 As oito tabelas

Tabela	Papel no domínio	Classe Java correspondente
T_USUARIO	Identidade do cliente SoulUp. O CPF é gravado apenas como hash, atendendo à LGPD.	Usuario
T_CARTEIRA	Saldo de Pontos ECOA. Relação 1:1 com o usuário, garantida por chave única.	Carteira
T_OPERADORA_TRANSPORTE	Catálogo das parceiras, com modal, valor da passagem e fator de CO_2 por km.	Operadora
T_FAIXA_CUPOM	Faixas fixas de valor e o custo em pontos de cada uma.	FaixaCupom
T_USUARIO_OPERADORA_FAVORITA	**Tabela associativa N:N** entre usuário e operadora, com atributos próprios (ordem de preferência e data).	UsuarioOperadoraFavorita
T_VOUCHER	O cupom — núcleo do produto. Guarda código, hash de validação, status e as datas de geração, expiração e uso.	Voucher
T_VIAGEM	Uso efetivo do cupom, com km estimado e CO_2 evitado. Relação 1:1 com o voucher.	Viagem
T_TRANSACAO_PONTOS	Log append-only de toda movimentação de pontos, com origem e IP.	TransacaoPontos

5.2 Decisões de modelagem que sustentam a API

- **Valores monetários em centavos (NUMBER inteiro).** Evita erro de arredondamento de ponto flutuante em dinheiro; a formatação em reais acontece na apresentação.
- **CO_2 gravado em gramas (NUMBER inteiro).** Mesma razão: soma exata no ranking e nos relatórios, conversão para kg só na saída.
- **Sequences explícitas em vez de triggers.** O id é obtido com NEXTVAL antes do INSERT, o que deixa a chave disponível na mesma transação para gravar as tabelas dependentes.
- **Extrato append-only.** T_TRANSACAO_PONTOS nunca sofre UPDATE nem DELETE de negócio: um estorno é um novo lançamento de crédito. É o que torna o saldo auditável.
- **Índice composto (id_usuario, status) em T_VOUCHER.** É exatamente a consulta que a regra antifraude faz a cada resgate; sem ele, a verificação varreria a tabela inteira.

6. Diagrama de classes atualizado

O projeto tem **54 classes distribuídas em 10 pacotes**. Para que cada diagrama fique legível, a estrutura está apresentada em camadas: primeiro a visão geral da arquitetura, depois as classes de domínio e, em seguida, cada camada técnica.

6.1 Visão geral da arquitetura

A aplicação é dividida em camadas com responsabilidade única. O front fala apenas com a camada **Resource**; a Resource não conhece SQL; a **BO** concentra as regras de negócio e o controle de transação; a **DAO** é a única que escreve SQL; e a **ConexaoOracle** é a fábrica de conexões, o padrão Connection Factory trabalhado em aula. A camada **Model** é usada por todas e não depende de nenhuma.

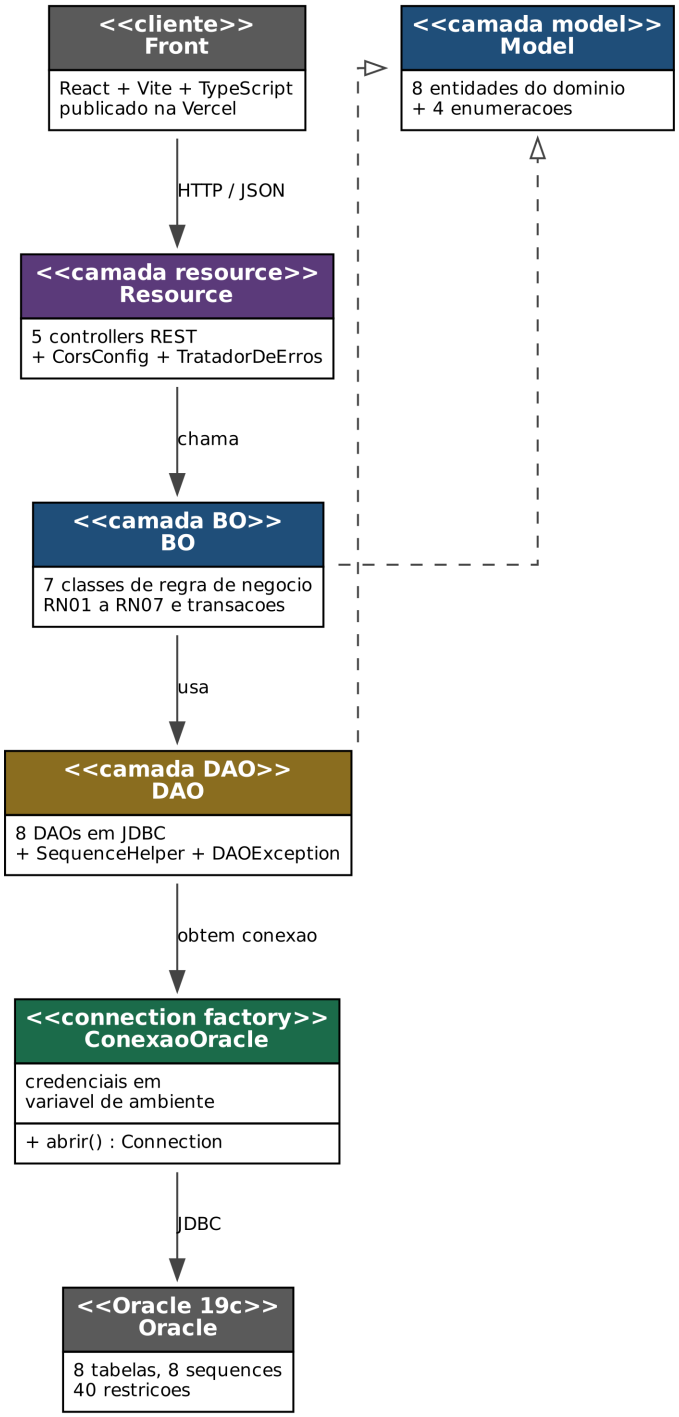


Figura 10 — Arquitetura em camadas. A seta cheia indica dependência de uso; a tracejada indica que a camada manipula as classes de modelo.

6.2 Camada Model — as classes de domínio

São 8 entidades e 4 enumerações, espelhando as 8 tabelas do MER. Todas seguem o mesmo padrão de encapsulamento: atributos privados, construtor padrão e construtor completo, getters e setters, e `equals`, `hashCode` e `toString` sobrescritos. O atributo identificador de toda entidade se chama simplesmente `id`; a coluna correspondente no banco mantém o nome completo (por exemplo `id_usuario`), e a tradução acontece no DAO.

Duas classes carregam comportamento além do estado: `Carteira`, com `debitar()` e `creditar()`, que já rejeitam saldo negativo; e `Voucher`, com `estaExpirado()`.

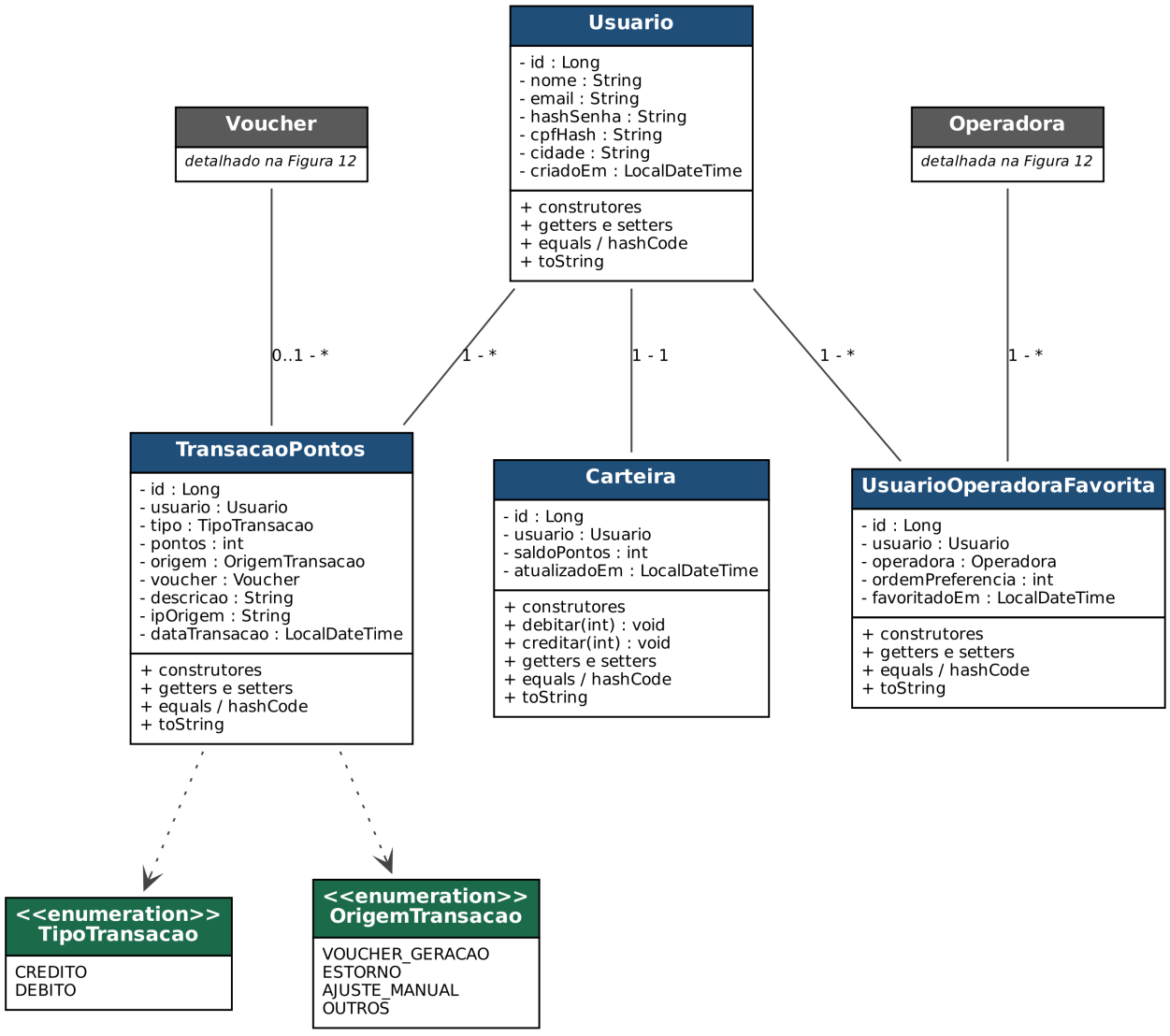


Figura 11 — Classes de domínio: usuário, carteira e movimentação de pontos. As caixas cinza referenciam classes detalhadas na figura seguinte.

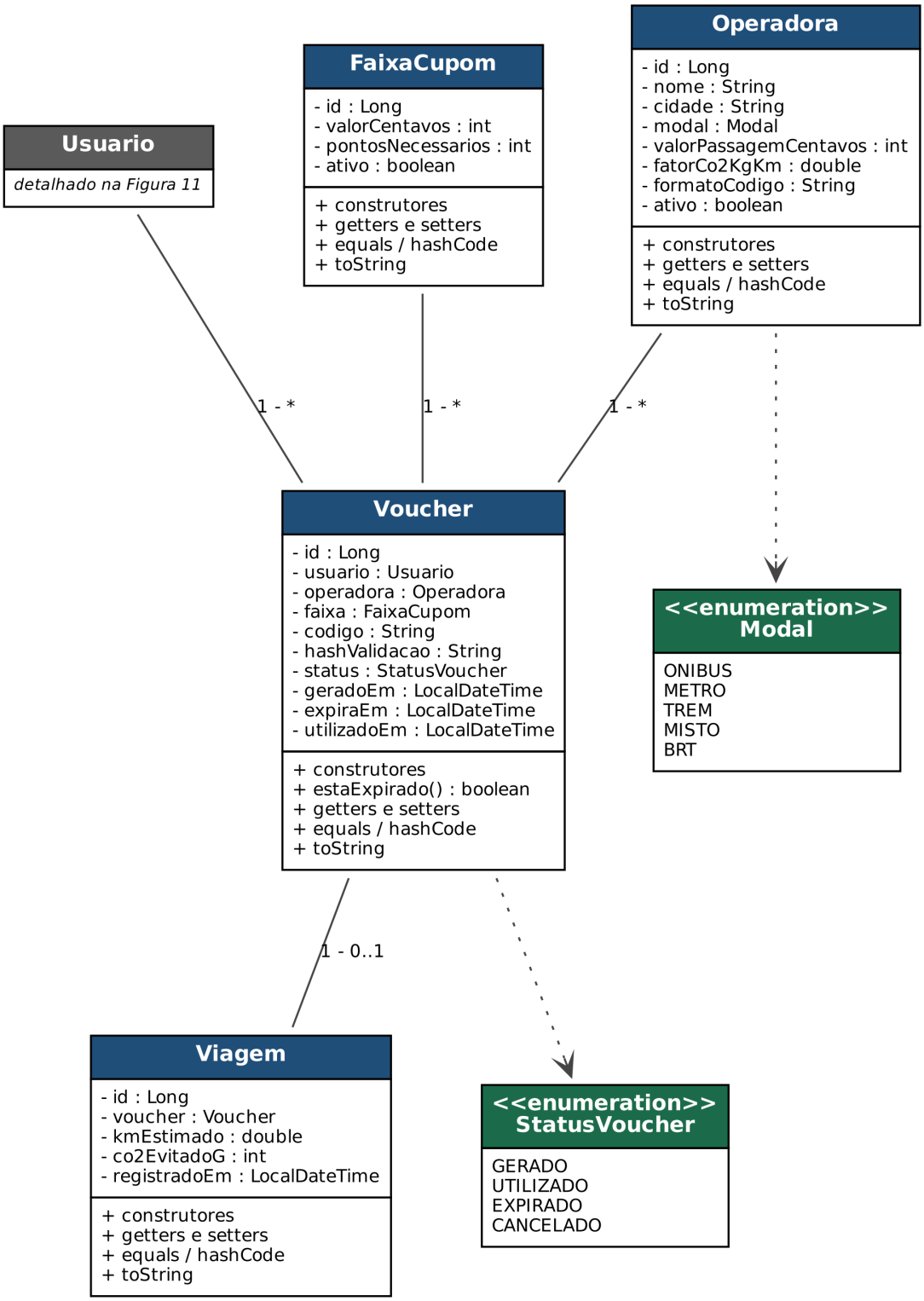


Figura 12 — Classes de domínio: cupom, operadora, faixa de valor e viagem.

6.3 Camada DAO e a Connection Factory

São 8 DAOs, um por entidade, todos anotados com `@Repository`. Cada um usa `PreparedStatement` e `ResultSet` diretamente — não há JPA nem Hibernate, justamente para demonstrar o padrão DAO.

Vários métodos aparecem em duas versões: uma que abre e fecha a própria conexão, usada em operações isoladas, e outra que recebe a `Connection` por parâmetro, usada quando o método participa de uma transação maior controlada pela BO. É esse par que permite, por exemplo, debitar a carteira, gravar o voucher e registrar o extrato como uma operação única.

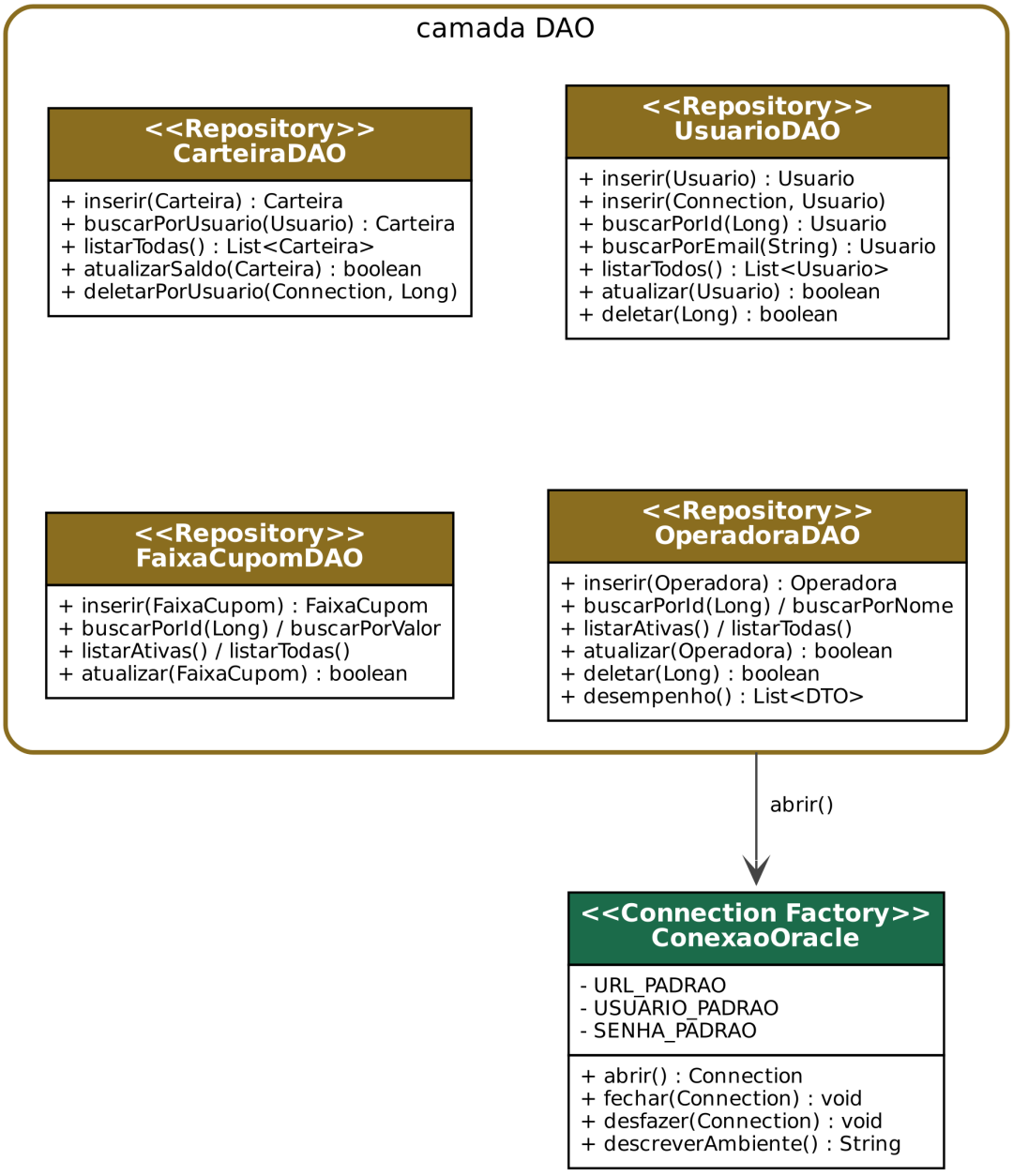


Figura 13 — Camada DAO (1 de 2) e a Connection Factory.

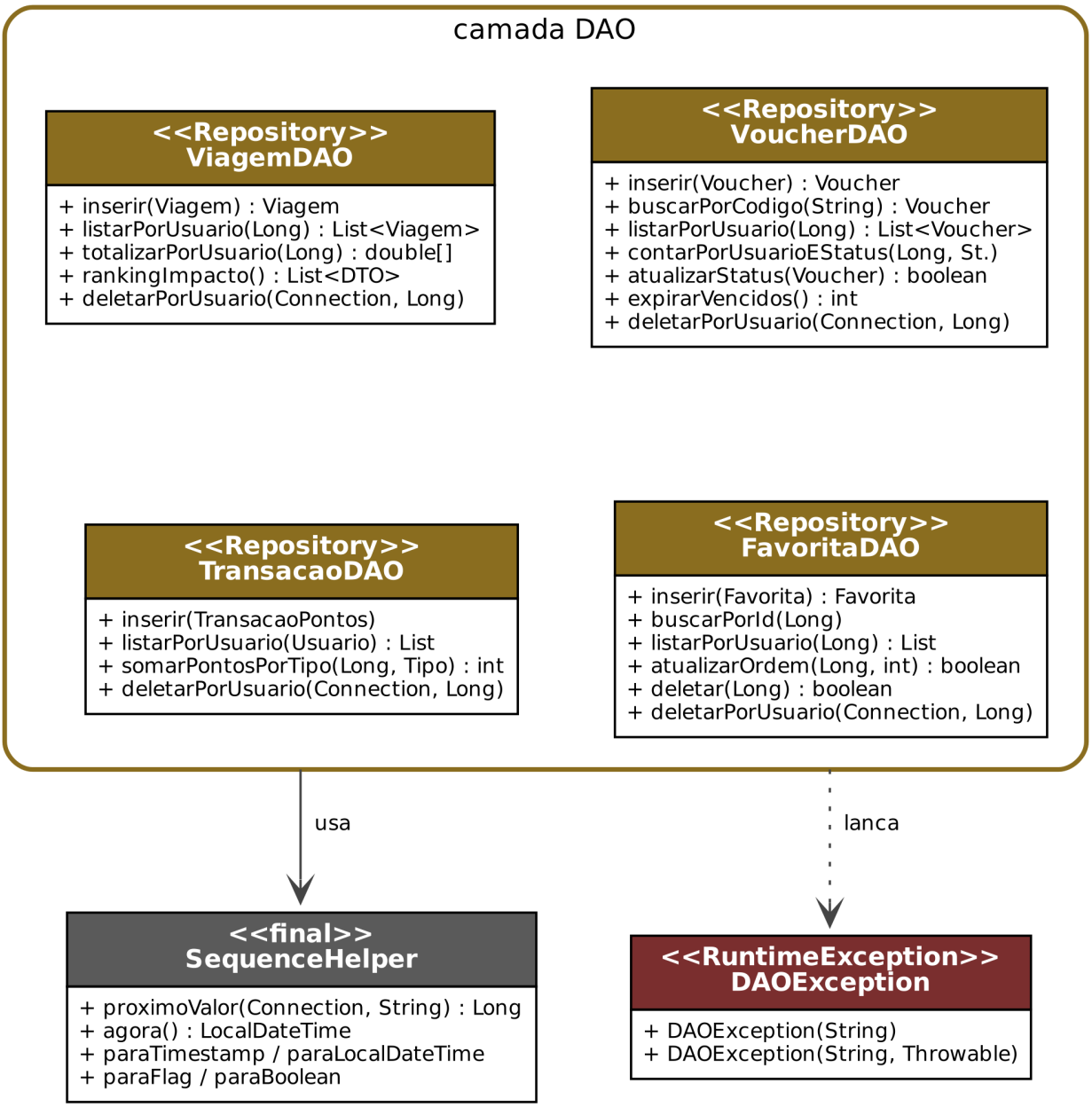


Figura 14 — Camada DAO (2 de 2) e as classes de apoio: o SequenceHelper, que obtém o próximo valor da sequence e converte tipos de data, e a DAOException, que encapsula qualquer SQLException.

6.4 Camada BO — as regras de negócio

São 7 classes anotadas com `@Service`, todas com injeção por construtor. O estereótipo em cada caixa indica quais regras de negócio aquela classe garante. A `ConversorBO` é a mais densa porque o resgate é a operação que toca mais tabelas: ela consulta a `AntiFraudeBO` antes de debitar e conduz toda a transação.

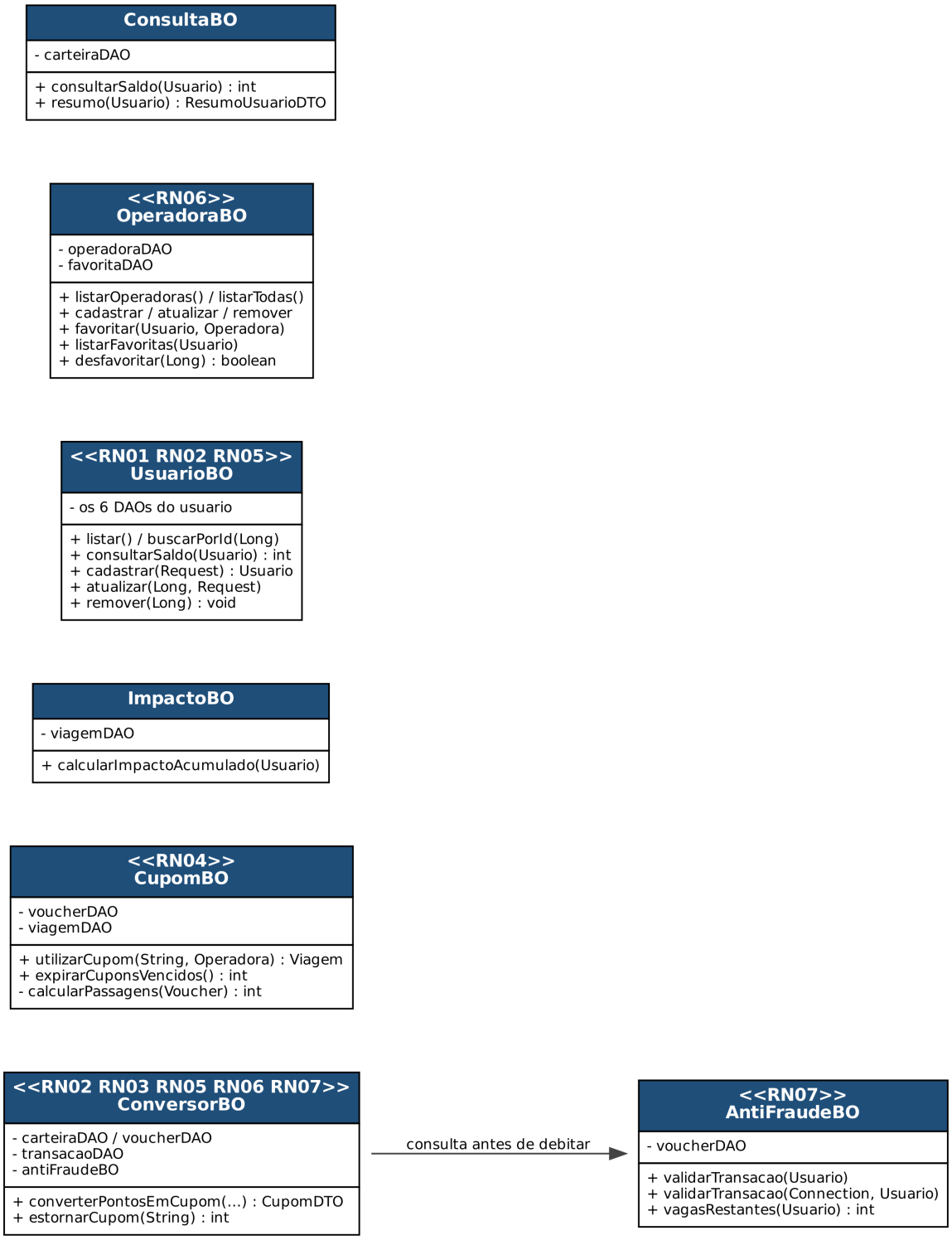


Figura 15 — Camada BO, com as regras de negócio que cada classe aplica.

6.5 Camada Resource, CORS e tratamento de erros

São 5 controllers anotados com `@RestController`. Eles não contêm regra de negócio: recebem a requisição, validam o formato da entrada, chamam a BO e montam a resposta com o status adequado.

A classe `CorsConfig` publica o bean `CorsFilter` exigido para que o front, hospedado em outro domínio, consiga chamar a API pelo navegador. A `TratadorDeErros`, anotada com `@RestControllerAdvice`, centraliza o tratamento de exceções descrito na seção 3.6.

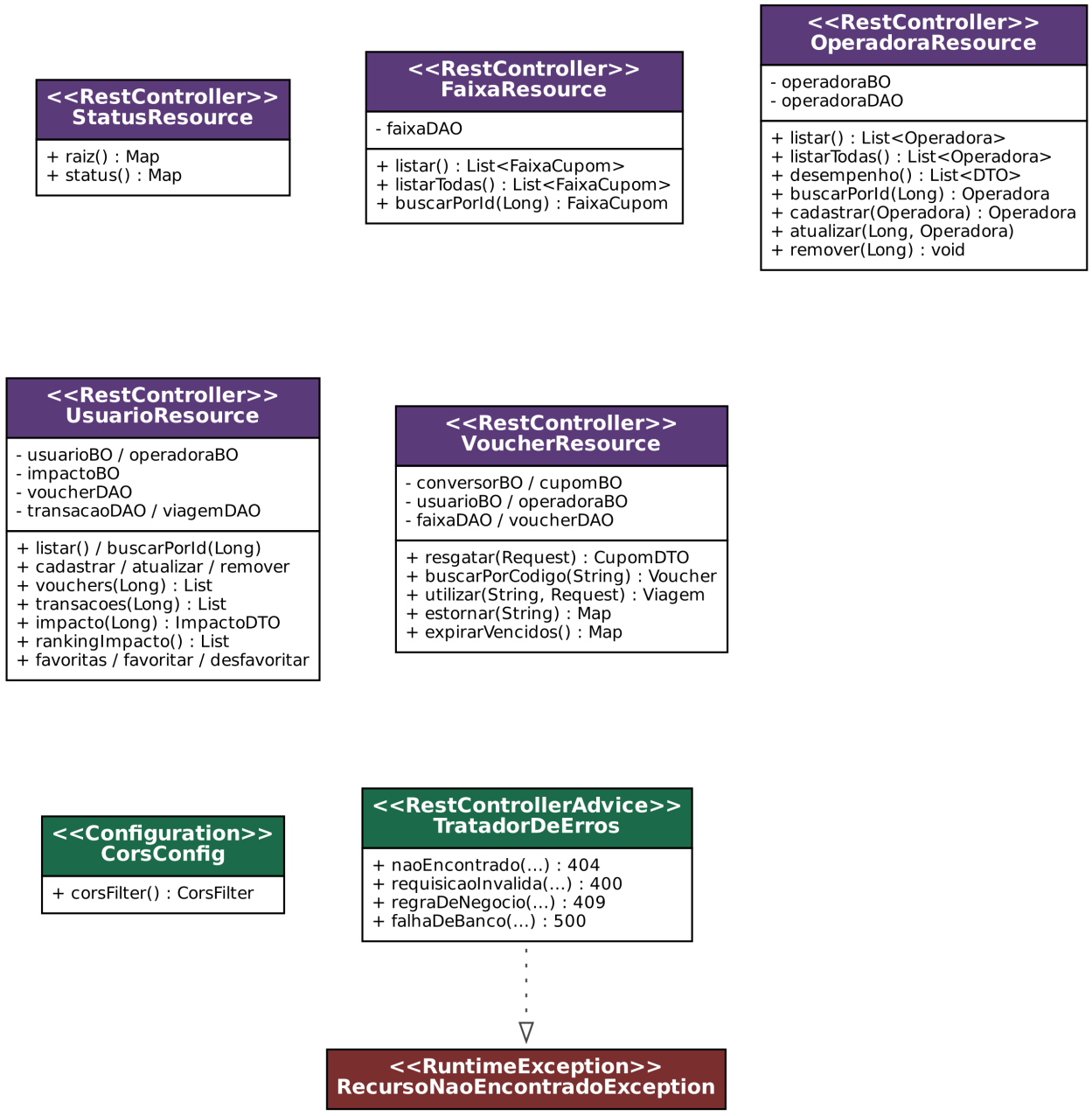


Figura 16 — Camada Resource, configuração de CORS e tratamento central de exceções.

6.6 Organização dos pacotes

Pacote	Classes	Responsabilidade
`model`	12	8 entidades de domínio e 4 enumerações
`dao`	10	8 DAOs, o SequenceHelper e a DAOException
`bo`	7	Regras de negócio e controle de transação
`resource`	5	Controllers REST
`dto`	12	Objetos de entrada e de saída da API
`util`	3	Hash, gerador de código de cupom e validadores
`exception`	2	Exceção de recurso não encontrado e tratador central
`conexao`	1	ConexaoOracle — a Connection Factory
`config`	1	CorsConfig — o bean CorsFilter
`(raiz)`	1	MobilidadeVerdeApplication — classe de inicialização

7. Procedimentos para rodar a aplicação

7.1 Ferramentas e versões

Item	Versão	Observação
Java JDK	17 ou superior	O projeto é compilado com `source` e `target` 17
Spring Boot	3.3.5	Framework da API Restful
Maven	3.9	Não precisa estar instalado: o projeto traz o Maven Wrapper
Oracle JDBC (ojdbc11)	23.5.0.24.07	Driver de conexão
Oracle Database	19c	Servidor da FIAP — `oracle.fiap.com.br:1521`, SID `ORCL`
IDE	IntelliJ IDEA Community	Também funciona em Eclipse ou VS Code com extensão Java

7.2 Pré-requisitos no banco

As oito tabelas precisam existir no schema, com os dados carregados. Os scripts `mobilidade_verde_ddl.sql` e `mobilidade_verde_carga.sql`, entregues na disciplina de Building Relational Database, fazem isso. No SQL Developer, abra cada script e execute com **F5** (Executar Script), na ordem: primeiro o DDL, depois a carga.

7.3 Execução pelo terminal

```
# 1. Clonar o repositório
git clone <url-do-repositorio>
cd mobilidade-verde-api

# 2. Compilar e empacotar
./mvnw clean package -DskipTests

# 3. Executar
java -jar target/mobilidade-verde-api-1.0.0.jar
```

No Windows, use `mvnw.cmd` no lugar de `./mvnw`. A aplicação sobe em `http://localhost:8080`. Para confirmar que subiu e que o banco está acessível:

```
curl http://localhost:8080/api/status
```

7.4 Importação no IntelliJ IDEA

- Abra o IntelliJ e escolha **File > Open**.
- Selecione o arquivo `pom.xml` da pasta do projeto e confirme com **Open as Project**.
- Aguarde o Maven baixar as dependências — a barra de progresso aparece no rodapé.
- Confirme em **File > Project Structure > Project** que o SDK é o Java 17 ou superior.
- Execute a classe `MobilidadeVerdeApplication`, no pacote `br.com.fiap.mobilidadeverde`, pelo botão verde ao lado do método `main`.

7.5 Credenciais de acesso ao banco

Por padrão a aplicação conecta no schema `RM569450`, com as credenciais gravadas na classe `ConexaoOracle` — assim o projeto roda logo após o clone, sem configuração. Para apontar para outro schema, basta definir as três variáveis de ambiente antes de executar; elas têm precedência sobre os valores do código:

```
export ORACLE_URL="jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL"
export ORACLE_USER="RM000000"
export ORACLE_PASSWORD="sua-senha"
```

No Windows (Prompt de Comando), troque `export` por `set`.

8. Deploy da aplicação

A API está publicada no **Render**, no plano gratuito. O repositório GitHub é público e o Render reconstrói a aplicação a cada push na branch principal.

O Render **não oferece runtime Java nativo** — a plataforma suporta nativamente apenas Node, Python, Ruby, Go, Rust e Elixir, e a própria documentação dela indica Docker para aplicações JVM. Por isso o repositório traz um `Dockerfile` com build em dois estágios: o primeiro compila o projeto com Maven e JDK 17, o segundo copia apenas o `.jar` pronto para uma imagem que contém somente a JRE. A imagem final fica bem menor e não carrega as ferramentas de compilação. O container roda com usuário sem privilégio de root.

O `Dockerfile` fica na **raiz do repositório**, e não dentro de `mobilidade-verde-api/`, porque o Render resolve tanto o caminho do Dockerfile quanto o contexto de build a partir da raiz do repositório. Com o arquivo nessa posição, nenhum campo de caminho precisa ser preenchido no painel.

8.1 Links da entrega

Recurso	Endereço
Repositório GitHub	`https://github.com/EstevesArtur/Mobilidade-Verde-JAVA-Sprint4`
API publicada (Render)	`https://mobilidade-verde-java-sprint4.onrender.com`
Verificação da API	`https://mobilidade-verde-java-sprint4.onrender.com/api/status`

8.2 Como o serviço foi configurado

Campo no Render	Valor
Language	`Docker`
Dockerfile Path	padrão (`./Dockerfile`)
Root Directory	em branco
Health Check Path	`/`
Instance Type	Free
Variável `ORACLE_URL`	`jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL`
Variável `ORACLE_USER`	O RM do aluno, em maiúsculas
Variável `ORACLE_PASSWORD`	A senha do aluno, cadastrada no painel do Render

Um serviço Docker não tem campo de *Build Command* nem de *Start Command*: quem define os dois é o `Dockerfile`.

O *health check* aponta para a raiz, e não para `/api/status`, de propósito. O servidor só considera o serviço no ar quando esse caminho responde 2xx, e `/api/status` devolve 503 quando o Oracle não responde. Se o health check dependesse do banco, uma oscilação no servidor da FIAP derrubaria a aplicação inteira, mesmo com a API funcionando. A raiz responde 200 sempre; o diagnóstico do banco continua disponível em `/api/status`, para quem quiser consultá-lo.

8.3 Como validar que o deploy está funcionando

Abra `<url-do-servico>/api/status` no navegador. A resposta esperada é:

```
{
  "aplicacao": "no ar",
  "url": "jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL",
  "usuario": "RM569450",
  "banco": "conectado",
  "detalhe": "Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production | usuario:
RM569450"
}
```

Essa é a resposta real do serviço publicado, conferida no navegador após o deploy. A leitura de dados foi confirmada em seguida por `/api/operadoras`, que devolveu as cinco operadoras ativas do catálogo direto do Oracle da FIAP.

Se o campo `banco` vier como `indisponivel` e a resposta for **503**, a aplicação subiu mas não alcançou o Oracle — nesse caso as variáveis de ambiente no painel do Render são o primeiro lugar a conferir.

Sobre a primeira requisição: no plano gratuito o Render hiberna o serviço depois de 15 minutos sem uso. A primeira chamada após esse período leva cerca de 50 segundos para responder, porque o contêiner precisa subir novamente. Não é falha da aplicação; a partir da segunda chamada o tempo de resposta volta ao normal.

8.4 Estrutura do projeto entregue

```
<repositório>/
├── Dockerfile                  imagem construída pelo Render
├── render.yaml               configuração do deploy
├── Documentacao_DDD_MobilidadeVerde_Sprint4.pdf
└── mobilidade-verde-api/
    ├── pom.xml                dependências e build
    ├── mvnw / mvnw.cmd      Maven Wrapper
    ├── README.md
    └── src/main/
        ├── resources/application.properties
        └── java/br/com/fiap/mobilidadeverde/
        ├── MobilidadeVerdeApplication.java
            ├── config/       1 classe
            ├── conexao/      1 classe
            ├── model/        12 classes
            ├── dao/          10 classes
            ├── bo/           7 classes
            ├── dto/          12 classes
            ├── resource/     5 classes
            ├── exception/    2 classes
            └── util/         3 classes
```

Equipe FOG · Turma 1TDSPX-2026 · FIAP
Representante: Artur Bianchini Esteves — RM569450
Challenge 2026 · SoulUp / Prospera · Sprint 4